

Módulo 06 — Orquestração com LangGraph.js

Estado, nós, arestas condicionais e reducers; checkpoint, threads, retomada e viagem no tempo; e quando usar as alternativas.

1. Visão geral
2. Conceitos
3. Exercício
4. Projeto final — etapa M6
5. Para revisar

Visão geral

Objetivos de aprendizagem

Ao final, a pessoa consegue:

1. modelar um fluxo como grafo de estado: estado, nós, arestas e arestas condicionais;
2. explicar o que um reducer faz e por que ele é necessário quando nós rodam em paralelo (fan-out com `Send`);
3. usar um checkpointer com threads para retomar uma execução depois de uma falha e para dar memória entre chamadas;
4. corrigir uma decisão passada com viagem no tempo (`getStateHistory` + `updateState`), sem apagar o histórico;
5. escolher, com critérios, entre LangGraph e as alternativas (código puro, Vercel AI SDK, Mastra, OpenAI Agents SDK, Claude Agent SDK).

Mensagem central

Um workflow vira **grafo** quando você precisa **retomar, desviar ou pausar**. Se o fluxo é curto e reto, uma função com `if` basta. E, seja qual for o framework, mantenha os papéis como funções simples: o framework só **liga** as peças.

Conceitos

No Módulo 5, o time do curador era um **pipeline em código**: uma função `montarEdicao` chamando os papéis em ordem, com um laço para o redator $\Rightarrow$ revisor. Funciona, mas tem três limites:

1. **Se o processo cair no meio, perde tudo.** O time levou ~14 minutos; uma falha no último passo obrigava a reclassificar as 32 notícias.
2. **O caminho é fixo.** Todo item passava pelo mesmo trajeto, com uma confiança alta ou baixa.
3. **É difícil ver o fluxo.** Para saber o que o código faz, é preciso ler o código.

Este módulo transforma o pipeline em um **grafo de estado** com o LangGraph.js: o fluxo vira dado (dá para desenhar), cada passo salva um checkpoint (dá para retomar) e os caminhos podem depender do estado (aresta condicional).

1. As quatro peças de um grafo

Peça	O que é	No curador
Estado	um objeto compartilhado, com campos tipados (os "canais")	<code>candidatos</code> , <code>avaliados</code> , <code>itens</code> , <code>paraRevisao</code> , <code>arquivo</code> ...
Nó	uma função que recebe o estado e devolve só o que mudou	<code>coletar</code> , <code>classificar</code> , <code>redigir</code> , <code>publicar</code>
Aresta	"depois deste nó, vá para aquele"	<code>redigir</code> $\rightarrow$ <code>publicar</code>
Aresta condicional	uma função que olha o estado e escolhe o próximo nó	<code>selecionar</code> $\rightarrow$ <code>redigir</code> ou $\rightarrow$ <code>fila_humana</code> , conforme a confiança

Analogia: um fluxograma de processo, só que executável. As caixas são os nós, as setas são as arestas, os losangos ("a confiança é $\geq 0,9$?") são as arestas condicionais, e o estado é a pasta que passa de mesa em mesa com os papéis do caso.

Reducers: como juntar atualizações

Quando um nó devolve `{ avaliados: [x] }`, o que acontece com o que já estava em `avaliados`? Depende do campo:

- **sem reducer:** o valor novo **substitui** o antigo (é o caso de `arquivo`);
- **com reducer:** o LangGraph **combina** o antigo com o novo. No projeto, o reducer de listas concatena.

Isso importa quando vários nós rodam **em paralelo** e escrevem no mesmo campo: sem `reducer`, um apagaria o resultado do outro. No curador, as 32 classificações rodam como 32 execuções do nó `classificar` (uma por notícia, disparadas com `Send`), e todas escrevem em `avaliados`.

No `LangGraph.js 1.x`, o estado é declarado com `StateSchema` e campos `Zod`; um campo com `reducer` usa `ReducedValue`. (Versões e exemplos antigos usam `Annotation.Root`, que continua funcionando.)

Fan-out com `Send`

Uma aresta condicional pode devolver uma **lista de `Send`**: cada `Send("classificar", { id })` agenda uma execução do nó com uma entrada própria. É o padrão "um para muitos" (*map*): uma tarefa por item, em paralelo, juntando no estado pelo `reducer`.

Duas armadilhas que encontramos

- **Aresta condicional logo depois de um fan-out roda uma vez por ramo.** Se cada `classificar` tivesse uma aresta condicional para "redigir ou não", ela rodaria 32 vezes, e os itens se repetiriam. A solução é um **nó de junção** (`selecionar`) ligado por aresta normal: o `LangGraph` espera todos os ramos terminarem e roda o nó uma vez, com o estado completo.
- **O nome de um nó não pode ser igual ao de um campo do estado.** O `LangGraph` recusa com um erro do tipo *"X is already being used as a state attribute"*. Por isso, no exercício, o campo é `atendidoPor` e os nós são `financeiro`, `suporte`...

2. Checkpoint: salvar a cada passo

Um **checkpointer** grava um retrato do estado (um *checkpoint*) depois de cada passo do grafo. Cada execução pertence a uma **thread**, identificada por `thread_id` na configuração:

```
const grafo = construtor.compile({ checkpointer: new MemorySaver() });
await grafo.invoke(entrada, { configurable: { thread_id: "edicao-da-semana" } });
```

Com isso, ganhamos:

Recurso	Como usar	Para que serve
Retomar depois de uma falha	<code>invoke(null, config)</code> na mesma thread	continua do último passo salvo, sem refazer o que já terminou
Memória entre chamadas	chamar de novo com o mesmo <code>thread_id</code>	o estado da thread continua lá (uma conversa, um cliente)

Recurso	Como usar	Para que serve
Inspeccionar	<code>getState(config)</code> , <code>getStateHistory(config)</code>	ver o estado atual, o próximo nó (<code>next</code>) e cada passo anterior
Viagem no tempo	<code>updateState(checkpointAntigo, valores, comoNo) + invoke(null, novoConfig)</code>	corrigir um valor no passado e seguir dali, criando um ramo novo, sem apagar o original
Pausar para um humano	<code>interrupt()</code> + <code>Command({ resume })</code>	o Módulo 7

Analogia: o "salvar jogo" de um videogame. Cada fase concluída grava o progresso; se a energia cair, você volta da última fase salva, não do começo. E dá para carregar um save antigo e jogar diferente, sem perder o original.

Onde o checkpoint fica

- `MemorySaver` : na memória do processo. Ótimo para testes; some quando o processo termina.
- O projeto traz `CheckpointEmArquivo` (um `MemorySaver` que grava um JSON), só para mostrar que um checkpoint é **dado**: dá para abrir o arquivo, matar o processo e retomar em outro.
- Em produção: um checkpointer de banco. Há pacotes oficiais para SQLite, Postgres e outros. No Módulo 8, discutimos o que usar num ambiente sem disco, como o Cloudflare Workers.

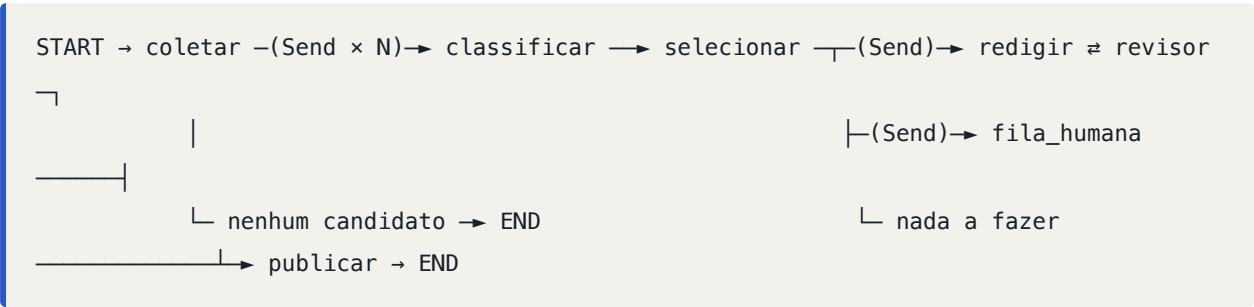
Execução durável: o que ela promete e o que não promete

Retomar de um checkpoint **reexecuta o nó que estava rodando** quando a falha aconteceu. Então:

- o que o nó fez **por fora** antes de falhar (enviar um e-mail, gravar um arquivo, cobrar um cartão) pode acontecer **duas vezes**;
- por isso, nós com efeito externo devem ser **idempotentes** (rodar duas vezes dá o mesmo resultado) ou separados em nós pequenos.

No curador, `publicar` falhou antes de gravar; na retomada, só ele rodou de novo.

3. O curador como grafo



- `coletar` → o agente da M5 (pelo MCP de fontes);
- `classificar` → um por notícia, em paralelo (`Send`);
- `selecionar` → nó de junção: aplica a regra editorial (limiar de confiança, até 10 itens, **no máximo 3 por categoria**: é a resposta ao viés da M5, em que a edição saiu sem regulação e sem mercado);
- `redigir` → um por item publicável; dentro dele, o ciclo redator $\rightleftharpoons$ revisor;
- `fila_humana` → itens com confiança baixa ou que o guardrail mandou revisar. Por enquanto só registra; **no Módulo 7, vira uma pausa de verdade** com aprovação humana;
- `publicar` → chama o servidor MCP de edição.

4. Alternativas ao LangGraph

LangGraph não é a única forma de orquestrar agentes. Uma comparação curta, para escolher com critério:

Opção	O que é	Pontos fortes	Quando escolher
Código puro (a M5)	funções, laços e <code>if</code>	zero dependência, fácil de testar, qualquer pessoa lê	fluxos curtos, sem necessidade de retomar ou pausar
Vercel AI SDK (o que usamos nas chamadas)	biblioteca para chamar modelos, com ferramentas, saída tipada e loop de agente	leve, vários provedores, ótimo para um agente	o agente é o produto; a orquestração é simples
LangGraph.js	grafo de estado com checkpoint	retomada, memória por thread, viagem no tempo, pausa humana, fluxo desenhável	fluxos longos, com ramos, falhas e humanos no meio
Mastra	framework TypeScript de agentes e <i>workflows</i> (<code>.then</code> , <code>.branch</code> , <code>.parallel</code>), com suspender e retomar	API encadeada mais "TypeScript", inclui memória, RAG e evals; licença Apache-2.0 (exceto os módulos enterprise)	quer um framework completo em TS, com workflows declarativos

Opção	O que é	Pontos fortes	Quando escolher
OpenAI Agents SDK (JS)	agentes com <i>handoffs</i> , <i>guardrails</i> , aprovação humana, sessões e tracing	o padrão handoff é nativo; funciona com outros provedores além da OpenAI	atendimento com transferência entre especialistas
Claude Agent SDK	o mesmo "motor" do Claude Code como biblioteca (TS/Python): ferramentas prontas de arquivo e terminal, <i>hooks</i> , subagentes, MCP, permissões	agente que trabalha num ambiente de arquivos e comandos, com controles de permissão	automações parecidas com um agente de código; usa os modelos Claude

Critérios para decidir:

1. **Precisa retomar, pausar ou voltar no tempo?** Sim → algo com checkpoint (LangGraph, Mastra). Não → código puro ou AI SDK.
2. **O fluxo é fixo ou o modelo decide os passos?** Fixo → workflow (grafo, Mastra, código). O modelo decide → um agente (AI SDK, OpenAI Agents SDK, Claude Agent SDK).
3. **Quanto a equipe aguenta de abstração?** Cada framework tem conceitos próprios para aprender e versões que mudam rápido.
4. **Prenda-se pouco:** mantenha os papéis (classificar, redigir...) como funções simples, e o framework só **liga** essas funções. Foi o que fizemos: o grafo da M6 reaproveita os papéis da M5 sem mudar uma linha deles.

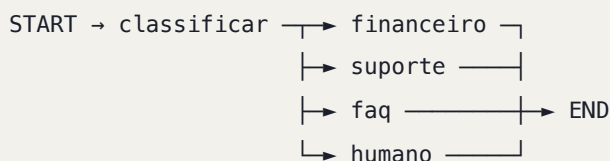
Referências

- LangGraph.js — visão geral: <https://docs.langchain.com/oss/javascript/langgraph/overview>
- LangGraph.js — Graph API (estado, nós, arestas, Send): <https://docs.langchain.com/oss/javascript/langgraph/graph-api>
- LangGraph.js — persistência (checkpointers, threads, getState, setState): <https://docs.langchain.com/oss/javascript/langgraph/persistence>
- LangGraph.js — execução durável: <https://docs.langchain.com/oss/javascript/langgraph/durable-execution>
- LangGraph.js — viagem no tempo: <https://docs.langchain.com/oss/javascript/langgraph/use-time-travel>
- Mastra — workflows: <https://mastra.ai/docs/workflows/overview>
- OpenAI Agents SDK (JS): <https://openai.github.io/openai-agents-js/>
- Claude Agent SDK: <https://code.claude.com/docs/en/agent-sdk/overview>
- Vercel AI SDK — agentes: <https://ai-sdk.dev/docs/agents/overview>
- Mermaid Live (para ver o desenho do grafo): <https://mermaid.live>

Exercício

Tarefa assíncrona. Os testes rodam offline: não precisa de modelo rodando.

Uma central de atendimento recebe chamados e precisa mandar cada um para o lugar certo: financeiro, suporte técnico, perguntas frequentes ou uma pessoa. Você vai montar isso como um **grafo de estado**:



O classificador é uma função injetada (nos testes, uma de mentira que decide por palavra-chave; na vida real, um LLM com saída tipada, como no Módulo 2). O que importa aqui é a **orquestração**: estado, nós, arestas condicionais e checkpoint.

Como rodar

Da raiz do repositório:

```
pnpm -F @mentoria/ex06-langgraph test          # versão base
pnpm -F @mentoria/ex06-langgraph test:desafio    # desafio extra
pnpm -F @mentoria/ex06-langgraph demo:grafo      # vê o grafo, os caminhos, a memória e
a viagem no tempo
pnpm -F @mentoria/ex06-langgraph demo:checkpoint # um processo morre no meio; outro
retoma
```

Os testes **falham** até você implementar. A solução de referência está em [solucao/](#). Tente antes de abrir. `pnpm -F @mentoria/ex06-langgraph test:solucao` roda os mesmos testes contra ela.

Versão base

Arquivo	O que fazer
<code>src/rotas.ts</code>	<code>rotaDeTriagem</code> : a função da aresta condicional (categoria + confiança → nó)
<code>src/triagem.ts</code>	<code>montarTriagem</code> : o grafo com <code>StateGraph</code> , os nós, as arestas e o checkpointer

Fornecidos em `src/estado.ts`: o estado (`EstadoTriagem`, com o `historico` acumulando por `reducer`), as categorias, os nós de atendimento e as respostas prontas.

Dicas:

- Um nó é uma função que recebe o estado e devolve **só o que mudou** (`{ categoria, confianca }`), não o estado inteiro.
- `addConditionalEdges("classificar", funcao, [destinos])`: a função devolve o nome do próximo nó. Passe a lista de destinos para o desenho do grafo sair certo.
- O nome de um nó **não pode** ser igual ao de um campo do estado (o `LangGraph` recusa). Por isso o campo é `atendidoPor` e os nós são `financeiro`, `suporte`...
- Sem `checkpointer` o grafo funciona, mas esquece tudo entre uma chamada e outra. Com ele, cada `thread_id` é uma conversa com memória.

Feito = `pnpm -F @mentoria/ex06-langgraph test` com todos os testes passando.

Desafio extra — viagem no tempo

O classificador mandou "Não recebi o reembolso" para o FAQ. Era cobrança. Em `src/desafio/viagem.ts`, implemente `corrigirCategoria`: ache no histórico de checkpoints o ponto logo depois da classificação, grave a categoria certa com `updateState` (criando um **ramo** novo) e continue dali com `invoke(null, config)`. Sem chamar o classificador de novo, e sem apagar o passado: o atendimento errado continua no histórico.

Feito = `pnpm -F @mentoria/ex06-langgraph test:desafio` com todos os testes passando.

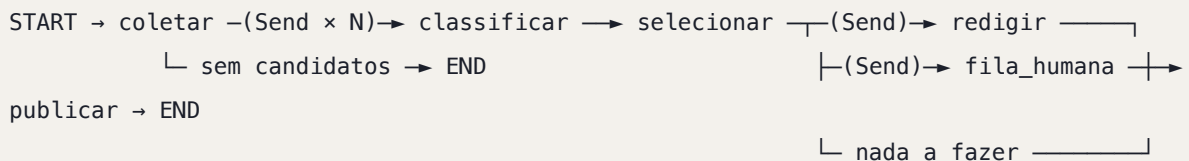
Perguntas para pensar

1. O que mudaria se `historico` fosse um campo simples, sem `reducer`? Teste e veja o que acontece com a memória da `thread`.
2. `rotaDeTriagem` é uma função pura. Que vantagem isso dá em relação a deixar o LLM escolher o próximo nó?
3. Onde, no seu trabalho, um processo precisa "lembrar" de interações anteriores com a mesma pessoa? O que seria o `thread_id`?
4. Viagem no tempo cria um ramo e mantém o passado. Para que serve guardar o caminho errado?

Projeto final — etapa M6

Etapa M6 — o curador como grafo (LangGraph.js)

Objetivo: o time da M5 vira um **grafo de estado**: classificações em paralelo, um caminho que depende da confiança, cota por categoria e **checkpoint** (se o processo cair, retoma de onde parou).



Implemente em `src/m06/`:

1. **rotas.ts**: as funções das arestas, puras e testáveis:
 - `enviarParaClassificacao`: um `Send("classificar", { id })` por candidato, ou `END`;
 - `selecionar`: a regra editorial (confiança $\geq$ limiar, até 10 itens, **no máximo 3 por categoria**; o resto vai para a revisão humana ou fica fora);
 - `rotearPorConfianca`: `Send` para `redigir` ou para `fila_humana`, ou `"publicar"` se não houver nada.
2. **grafo.ts**: `montarGrafo`, com os nós `coletar`, `classificar`, `selecionar` (junção), `redigir` (o ciclo redator $\Leftrightarrow$ revisor), `fila_humana` e `publicar`, compilado com o `checkpoint` recebido.

Fornecidos: `estado.ts` (o estado, com reducers que concatenam listas) e `checkpoint-arquivo.ts` (um `checkpoint` que grava em JSON). Os papéis são os mesmos da M5, sem mudança.

Feito =

1. `pnpm -F @mentoria/curador exec vitest run test/m06.test.ts verde` (offline, com papéis de mentira);
2. o grafo roda com o modelo de verdade, cai de propósito no `publicar` e retoma do `checkpoint`:

```
pnpm -F @mentoria/curador m06:grafo -- --nova --falhar # simula o servidor de
edição fora do ar
pnpm -F @mentoria/curador m06:grafo # retoma: só o publicar
roda de novo
```

Grava `saidas/m06-relatorio.md`, `saidas/m06-grafo.mmd` (cole em <https://mermaid.live>) e `saidas/m06-checkpoint.json`.

Responda (vamos discutir na Aula 7):

- Na retomada, quantas chamadas de cada papel aconteceram? Por quê?
- A cota por categoria mudou a cara da edição em relação à M5?
- Quais itens foram para a fila humana, e por quê? O que você faria com eles? (É o assunto da Aula 7.)

Desafio extra (seleção justa): na execução de referência, o modelo deu 0,95 a todos os itens de `mercado` e 0,98 aos das outras categorias, e mercado ficou fora da edição mesmo com a cota. Mude `selecionar` para alternar as categorias (a melhor de cada, depois a segunda de cada...) e acrescente um teste que prove que toda categoria com item publicável entra na edição.

Desafio extra (checkpoint de banco): troque `CheckpointEmArquivo` por um checkpoint de banco (`@langchain/langgraph-checkpoint-sqlite`) e repita a falha e a retomada.

Para revisar

Perguntas para discussão (15 min)

1. O curador precisava de um grafo, ou o pipeline da Aula 5 bastava? O que pesou na decisão?
2. Que processo do seu trabalho cai no meio e hoje recomeça do zero? Onde ficaria o checkpoint?
3. Onde um processo precisa lembrar de interações anteriores com a mesma pessoa? O que seria a thread?
4. A cota de 3 por categoria é uma regra editorial escrita no código. Quem deveria decidir essa regra, e como ela deveria mudar?
5. Para que serve guardar no histórico o caminho que estava errado?

Armadilhas comuns

- **Usar um framework de grafo para um fluxo curto e reto.** Mais conceitos, mais dependências, nenhum ganho.
- **Campo de lista sem reducer** com nós em paralelo: cada nó apaga o que o outro escreveu.
- **Aresta condicional logo depois de um fan-out:** roda uma vez por ramo e duplica o trabalho, sem dar erro. Use um nó de junção.
- **Nó com o mesmo nome de um campo do estado:** o LangGraph recusa.
- **Nó devolvendo o estado inteiro** em vez de só o que mudou: com reducers, duplica as listas.
- **Esquecer o `thread_id`** (ou gerar um novo a cada chamada): sem ele, não há retomada nem memória.
- **Efeito externo não idempotente num nó:** retomar reexecuta o nó que falhou, e o e-mail sai duas vezes.
- **MemorySaver em produção:** some quando o processo termina. Use um checkpointer de banco.
- **Deixar o LLM escolher o próximo nó** quando uma regra no código resolve: a rota fica imprevisível e difícil de testar.